Table des matières

Labo 04 OLTP — Fonctions de fenêtrage sur le transactionnel 1
Le terrain, en une minute 1
Partie 1 — Voir la différence : GROUP BY contre fenêtre 2
Partie 2 — Les grandes familles 2
Partie 3 — Ce que le contraste vous a montré 5
Et ensuite ? 5

Labo 04 OLTP — Fonctions de fenêtrage sur le transactionnel

Laboratoire du Cours 04, voie OLTP · PostgreSQL + AdventureWorks OLTP · Formation SQL avancé

On refait le laboratoire OLAP, question pour question, mais sur l'autre terrain : la base transactionnelle `Adventureworks` dans PostgreSQL. Connectez-vous avec pgAdmin, DBeaver ou `psql`. Comptez une demi-heure.

Les deux voies — Ce labo est le jumeau de `Labo_04_OLAP_SQL.md`. Les questions métier sont les mêmes ; le schéma, lui, ne l'est pas. Gardez les deux fichiers ouverts côte à côte : c'est en comparant deux requêtes qui répondent à la même chose que le contraste devient visible.

Identifiants en minuscules — Le portage PostgreSQL d'AdventureWorks range les tables dans des schémas en minuscules (`production`, `sales`, `person`) et replie les noms de colonnes de la même façon : `ListPrice` s'écrit `listprice`. Si une requête copiée d'ailleurs échoue sur un nom de colonne, c'est presque toujours ça.

Le terrain, en une minute

Là où l'entrepôt vous donnait un fait entouré de dimensions, le transactionnel vous donne des tables normalisées, plus nombreuses et plus étroites. Les quatre qui servent ici :

Table	Ce qu'elle contient	Grain
`sales.salesorderheader`	l'en-tête d'une commande : client, dates, totaux	une commande
`sales.salesorderdetail`	les lignes d'une commande : produit, quantité, prix	une ligne de commande

© 2026 iAlexMG. Tous droits réservés.

Table	Ce qu'elle contient	Grain
`production.product`	le catalogue : nom, couleur, prix de liste, sous-catégorie	un produit
`production.productsubcategory`	le libellé des sous-catégories	une sous-catégorie

Première différence à enregistrer : il n'y a pas de table de dates. La date vit dans la colonne `orderdate` de l'en-tête, et c'est à vous de la découper.

```sql
SELECT COUNT(*) FROM sales.salesorderdetail;   -- 121 317 lignes
SELECT COUNT(*) FROM production.product;       -- 504 produits
```

Partie 1 — Voir la différence : `GROUP BY` contre fenêtre

Théorie : `Theorie_04_SQL.md`, sections 1 et 2.

D'abord le `GROUP BY`, un total par produit, une ligne par produit :

```sql
SELECT productid, SUM(linetotal) AS total_produit
FROM sales.salesorderdetail
GROUP BY productid
ORDER BY total_produit DESC;
```

Maintenant la fenêtre : le même total, mais chaque ligne de commande reste visible.

```sql
SELECT salesorderid, productid, linetotal,
       SUM(linetotal) OVER (PARTITION BY productid) AS total_produit
FROM sales.salesorderdetail
ORDER BY productid
LIMIT 200;
```

Observez. Le premier résultat a une ligne par produit ; le second garde toutes les lignes de commande et répète le total en face de chacune. Même calcul, deux usages, et c'est rigoureusement le même constat que sur l'entrepôt. La syntaxe n'a pas bougé d'un caractère : seul le nom des tables a changé.

Le `LIMIT 200` n'est pas décoratif. Sur une base transactionnelle, on prend l'habitude de borner ce qu'on ramène, parce que la machine est souvent partagée avec l'application qui sert les clients.

Partie 2 — Les grandes familles

2.1 — Classer : `ROW_NUMBER`, `RANK`, `DENSE_RANK`

Théorie : `Theorie_04_SQL.md`, section 3.

© 2026 iAlexMG. Tous droits réservés.

Objectif : classer les produits du plus cher au moins cher, et voir comment les égalités sont traitées.

```sql
SELECT name, listprice,
       ROW_NUMBER() OVER (ORDER BY listprice DESC) AS no_ligne,
       RANK()       OVER (ORDER BY listprice DESC) AS rang,
       DENSE_RANK() OVER (ORDER BY listprice DESC) AS rang_dense
FROM production.product
WHERE listprice > 0
ORDER BY listprice DESC;
```

Observez. Là où plusieurs produits ont le même prix, `ROW_NUMBER` donne des numéros tous différents, `RANK` répète le rang puis saute, `DENSE_RANK` répète sans sauter. Identique à l'entrepôt, au nom des colonnes près.

2.2 — Top N par groupe, sans `QUALIFY`

Théorie : `Theorie_04_SQL.md`, sections 3 et 7.

Objectif : les 3 produits les plus chers de chaque sous-catégorie, en une seule requête.

C'est ici que les deux voies divergent pour de bon. `QUALIFY` n'existe pas dans PostgreSQL. Il faut donc calculer la fenêtre dans une sous-requête, puis filtrer à l'extérieur.

```sql
SELECT productsubcategoryid, name, listprice
FROM (
    SELECT productsubcategoryid, name, listprice,
           ROW_NUMBER() OVER (PARTITION BY productsubcategoryid
                              ORDER BY listprice DESC, productid) AS rang
    FROM production.product
    WHERE listprice > 0 AND productsubcategoryid IS NOT NULL
) AS classe
WHERE rang <= 3
ORDER BY productsubcategoryid, listprice DESC;
```

Observez. L'emboîtement dit pourquoi il est là : on ne peut pas filtrer sur une fonction de fenêtre dans le `WHERE`, parce que le `WHERE` s'exécute avant que les fenêtres soient calculées. C'est l'ordre d'exécution du Cours 01 qui commande, pas un caprice de dialecte. `QUALIFY`, du côté de l'entrepôt, n'est rien d'autre qu'un raccourci qui écrit cette sous-requête à votre place : la même requête, la même fenêtre, la même leçon.

Le résultat, lui, n'est pas le même des deux côtés, et c'est la vraie leçon du double chemin. Ici, 93 lignes ; sur l'entrepôt, 92, dont deux noms qui reviennent, « HL Road Handlebars » deux fois et « AWC Logo Cap » trois fois. Ce n'est pas un écart de dialecte : c'est un écart de modèle. `production.product` porte 504 produits, un par produit. `dbo.DimProduct` en porte 606 pour les mêmes 504 noms, parce qu'un produit dont le prix change y reçoit une nouvelle ligne, avec ses dates de validité. Une dimension à historique compte ses versions, pas ses entités, et un top N y classe donc des versions. Le Cours 05 revient sur ce mécanisme avec `DimEmployee` ; retenez pour l'instant que « le même SQL sur les deux terrains » ne veut pas dire « le même résultat ».

© 2026 iAlexMG. Tous droits réservés.

Changez `<= 3` en `= 1` : vous n'avez plus que le champion de chaque sous-catégorie.

2.3 — Regarder le mois précédent : `LAG`

Théorie : `Theorie_04_SQL.md`, sections 4 et 8. Théorie : `Theorie_02_SQL.md`, sections 4 et 7.

Objectif : les ventes par mois, avec la variation par rapport au mois précédent.

Seconde divergence, et elle est structurelle. L'entrepôt avait une `DimDate` où l'année et le mois étaient des colonnes prêtes à l'emploi. Ici, il n'y a qu'un horodatage, `orderdate`, qu'il faut découper soi-même.

```
SELECT annee, mois, ventes,
       LAG(ventes) OVER (ORDER BY annee, mois)          AS ventes_mois_prec,
       ventes - LAG(ventes) OVER (ORDER BY annee, mois) AS variation
FROM (
    SELECT EXTRACT(YEAR  FROM h.orderdate)::int AS annee,
           EXTRACT(MONTH FROM h.orderdate)::int AS mois,
           SUM(d.linetotal)                     AS ventes
    FROM sales.salesorderheader AS h
    JOIN sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
    GROUP BY 1, 2
) AS mensuel
ORDER BY annee, mois;
```

Observez. La première ligne a `ventes_mois_prec` à `NULL`, comme sur l'entrepôt. Mais comparez les deux sous-requêtes : côté entrepôt, une jointure vers `DimDate` et deux colonnes déjà taillées ; côté transactionnel, un `EXTRACT` sur un horodatage et une jointure de plus pour rejoindre les lignes à leur en-tête. La question est identique, le chemin non.

Le `::int` force le type : `EXTRACT` renvoie un `numeric` dans PostgreSQL, vérifié par `pg_typeof`, alors que l'année et le mois de l'entrepôt sont des entiers. Ce `numeric` s'affiche `2024` dans `psql`, sans décimales : le forcer ne change rien à l'œil, mais il donne une colonne du même type que celle de l'entrepôt, et épargne une conversion le jour où l'on compare ou réunit les deux.

2.4 — Cumul, moyenne mobile et part du total

Théorie : `Theorie_04_SQL.md`, sections 5 et 6.

Objectif : sur ces mêmes ventes mensuelles, ajouter un cumul, une moyenne mobile sur 3 mois et la part du total.

```
SELECT annee, mois, ventes,
       SUM(ventes) OVER (ORDER BY annee, mois)          AS cumul,
       AVG(ventes) OVER (ORDER BY annee, mois
                         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS moy_mobile_3,
       ventes / SUM(ventes) OVER ()                     AS part_du_total
```


© 2026 iAlexMG. Tous droits réservés.

```sql
FROM (
    SELECT EXTRACT(YEAR  FROM h.orderdate)::int AS annee,
           EXTRACT(MONTH FROM h.orderdate)::int AS mois,
           SUM(d.linetotal)                     AS ventes
    FROM sales.salesorderheader AS h
    JOIN sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
    GROUP BY 1, 2
) AS mensuel
ORDER BY annee, mois;
```

Observez. `cumul` monte sans cesse, `moy_mobile_3` lisse les à-coups, et `part_du_total` rapporte chaque mois au total des 38 mois : la fenêtre vide, `OVER ()`, couvre tout le résultat, et la colonne additionnée donne exactement 1. Enlevez le `ROWS BETWEEN` de la moyenne : elle devient un cumul divisé, ce qui n'est pas ce qu'on veut. Le cadre compte, et il compte pareil sur nos deux terrains : c'est du SQL standard, pas une particularité de moteur. Cela vaut pour les cadres eux-mêmes ; trois fonctions qui en dépendent, `FIRST_VALUE`, `LAST_VALUE` et `NTH_VALUE`, ne reçoivent pas le même cadre par défaut partout, et la théorie le dit à sa section 4.

Partie 3 — Ce que le contraste vous a montré

Théorie : `Theorie_04_SQL.md`, section 8.

Prenez une minute pour poser les trois écarts que vous venez de rencontrer. Ils reviendront à chaque cours.

Ce qui a changé	Voie OLAP (entrepôt)	Voie OLTP (transactionnel)
Retrouver le mois d'une vente	une jointure vers `DimDate`, colonnes prêtes	`EXTRACT` sur `orderdate`, à découper soi-même
Relier une vente à son montant	tout est dans `FactInternetSales`	jointure en-tête et lignes obligatoire
Filtrer sur une fenêtre	`QUALIFY`, une ligne	sous-requête puis `WHERE`, quatre lignes
Ce qui n'a pas changé	`OVER`, `PARTITION BY`, les cadres, les fonctions de classement, `LAG` et `LEAD`	idem, au caractère près

La colonne du bas est celle qui compte : le langage est le même. Ce qui vous ralentit en passant d'un monde à l'autre, ce n'est jamais la syntaxe des fenêtres, c'est la forme du schéma et la présence ou l'absence d'une dimension temps.

Les exercices du cours et leur corrigé vivent dans `Exercices_corriges_04_SQL.md`. Faites-les après avoir parcouru les deux labos.

Et ensuite ?

© 2026 iAlexMG. Tous droits réservés.

Vous venez de voir la même mécanique tenir sur deux schémas opposés. Les requêtes commencent à s'allonger, ici plus encore que sur l'entrepôt, à cause des jointures et des sous-requêtes imbriquées : le Cours 05 introduit les CTE (`WITH`) pour les rendre enfin lisibles.

© 2026 iAlexMG. Tous droits réservés.